

Java 15 — Text Blocks Final

Java Version: 15

Year: 2020

Feature: Text Blocks (*Final*)

UniqueClassName: TextBlocksFinal

Definition

Text Blocks became a **permanent Java language feature in Java 15** after being introduced as a preview feature in Java 13 and refined in Java 14.

A text block is a multi-line string literal enclosed by **three double quotes** (`"""`).

It makes it much easier to represent structured multi-line text such as:

- JSON
- XML
- HTML
- SQL
- Multi-line messages
- Configuration data

Traditional String

Before text blocks:

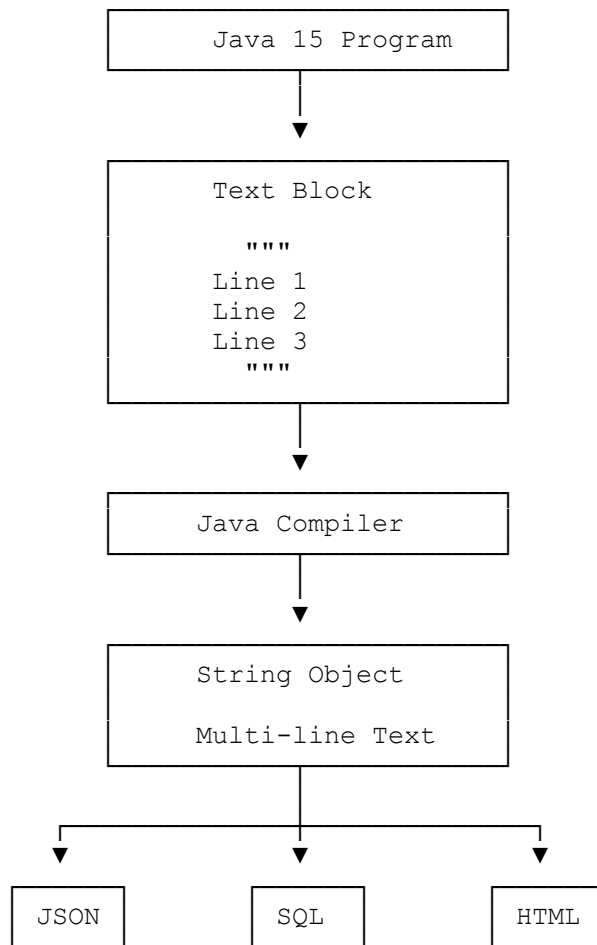
```
String json = "{\n" +  
    "  \"name\": \"Java\", \n" +  
    "  \"version\": 15\n" +  
    "}";
```

Java 15 Text Block

```
String json = """  
    {  
      "name": "Java",  
      "version": 15  
    }  
    """;
```

The second version is more readable because the Java source code closely resembles the resulting text.

Flowchart



Java Code Example

TextBlocksFinal.java

```
public class TextBlocksFinal {  
    public static void main(String[] args) {  
        String message = \"\"\"  
        Welcome to Java 15!  
        Text Blocks are now a  
        permanent language feature.  
        \"\"\";  
  
        System.out.println(message);  
    }  
}
```

Output

Welcome to Java 15!
Text Blocks are now a
permanent language feature.

Unlike Java 13, **no --enable-preview option is required** in Java 15.

JSON Example

Text blocks are particularly useful for JSON.

```
public class TextBlocksFinal {  
    public static void main(String[] args) {  
        String json = ""  
            {  
                "id": 101,  
                "name": "Anurag",  
                "language": "Java",  
                "version": 15  
            }  
            "";  
        System.out.println(json);  
    }  
}
```

Output:

```
{  
  "id": 101,  
  "name": "Anurag",  
  "language": "Java",  
  "version": 15  
}
```

SQL Example

```
public class TextBlocksFinal {  
    public static void main(String[] args) {  
        String sql = ""  
            SELECT id, name, email  
            FROM customer  
            WHERE status = 'ACTIVE'  
            ORDER BY name;  
    }  
}
```

```

        """;
    System.out.println(sql);
}
}

```

Output:

```

SELECT id, name, email
FROM customer
WHERE status = 'ACTIVE'
ORDER BY name;

```

HTML Example

```

public class TextBlocksFinal {

    public static void main(String[] args) {

        String html = ""
            <html>
                <body>
                    <h1>Java 15</h1>
                    <p>Text Blocks</p>
                </body>
            </html>
            """;

        System.out.println(html);
    }
}

```

This is considerably easier to read than a conventional Java string containing numerous `\n`, `\`, and concatenation operators.

Text Block Formatting

Java text blocks provide useful control over whitespace.

```

public class TextBlocksFinal {

    public static void main(String[] args) {

        String text = ""
            Java
            is
            powerful
            """;
    }
}

```

```

        System.out.println(text);
    }
}

```

Conceptually:

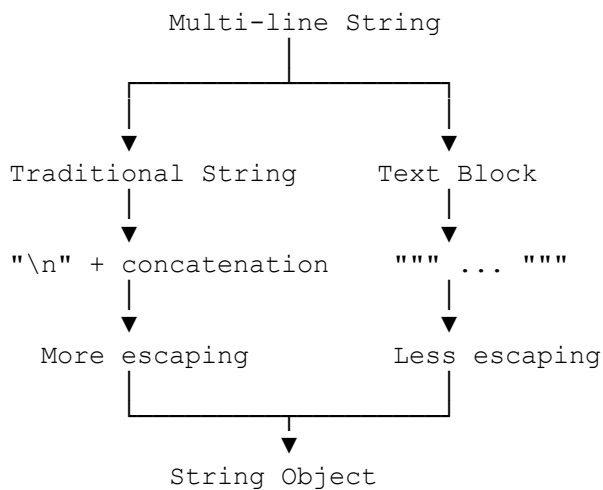
```

"""                                ← Opening delimiter
    Java
    is
    powerful
"""                                ← Closing delimiter

```

The Java compiler handles incidental indentation so that the indentation used to format the source code does not unnecessarily become part of the resulting string.

Text Block vs Normal String



Traditional

```

String text = "Line 1\\n"
              + "Line 2\\n"
              + "Line 3";

```

Java 15

```

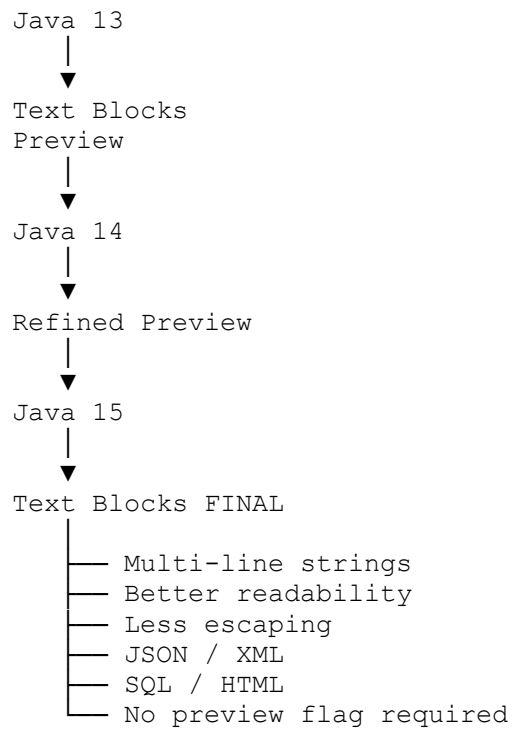
String text = """
    Line 1
    Line 2
    Line 3
    """;

```

Important Characteristics

Feature	Text Blocks in Java 15
Introduced	Java 13
Java 13 status	Preview
Java 14 status	Second preview
Java 15 status	Final/Permanent
Syntax	""" ... """
Primary purpose	Multi-line strings
--enable-preview required?	No
Useful for	JSON, XML, SQL, HTML, etc.

Key Point



In short: Java 15 finalized **Text Blocks**, making `""" ... """` a standard Java feature for writing readable multi-line strings without the excessive escaping and concatenation traditionally required.

Java 15 — Hidden Classes

Java Version: 15

Year: 2020

Feature: Hidden Classes

UniqueClassName: HiddenClasses

Definition

Hidden Classes were introduced as a standard feature in **Java 15** through **JEP 371**.

A hidden class is a class that is intended to be used **indirectly by frameworks and runtime-generated code**, rather than being directly referenced by ordinary application code.

They are particularly useful for:

- Dynamic language implementations
- Runtime code generation
- Frameworks
- Reflection-based libraries
- Lambda/metaprogramming mechanisms
- Classes that should not be discoverable or reused like ordinary application classes

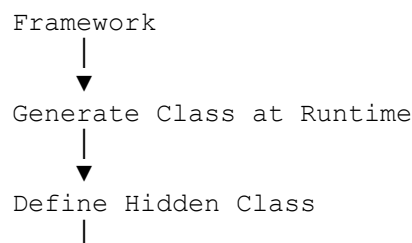
Hidden classes are closely associated with the **MethodHandles.Lookup API**.

The key API is:

`MethodHandles.Lookup.defineHiddenClass(...)`

Why Hidden Classes?

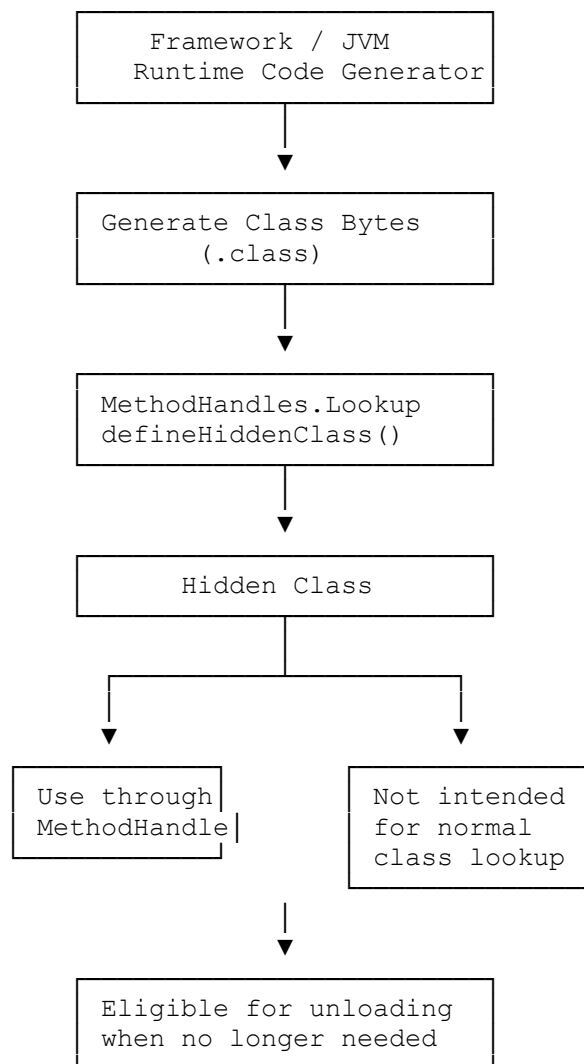
Suppose a framework dynamically generates a class:



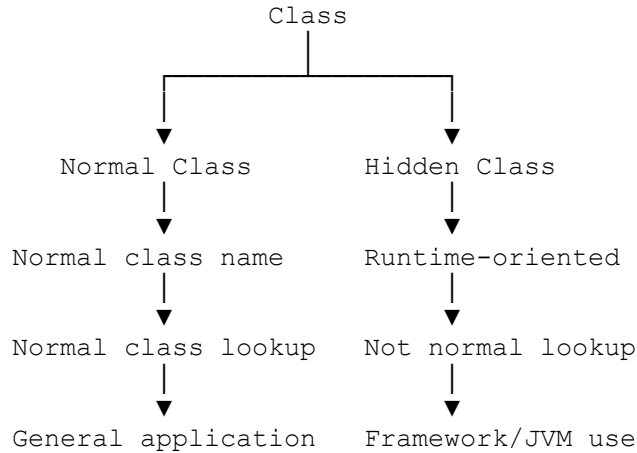


Unlike a normal class, a hidden class is not intended to become part of the application's ordinary class namespace.

Flowchart



Normal Class vs Hidden Class



Hidden classes are therefore especially useful for **runtime-generated implementation classes**.

Java Code Example

A simple demonstration can define a hidden class from the bytecode of another class.

HiddenClasses.java

```
import java.lang.invoke.MethodHandles;

public class HiddenClasses {

    public static void main(String[] args) throws Throwable {

        // Obtain a Lookup object
        MethodHandles.Lookup lookup =
            MethodHandles.lookup();

        // Read bytecode of the nested class
        byte[] classBytes = getClassBytes();

        // Define the class as a hidden class
        MethodHandles.Lookup hiddenLookup =
            lookup.defineHiddenClass(
                classBytes,
                true
            );

        // Get the hidden class
        Class<?> hiddenClass =
            hiddenLookup.lookupClass();

        System.out.println(
```

```

        "Hidden class: " + hiddenClass
    );

    System.out.println(
        "Is hidden: " + hiddenClass.isHidden()
    );
}

private static byte[] getClassBytes() throws Exception {

    String resourceName =
        HiddenTarget.class.getName()
            .replace('.', '/')
            + ".class";

    try (var inputStream =
        HiddenClasses.class
            .getClassLoader()
            .getResourceAsStream(resourceName)) {

        if (inputStream == null) {
            throw new IllegalStateException(
                "Class file not found"
            );
        }

        return inputStream.readAllBytes();
    }
}

static class HiddenTarget {

    public void hello() {
        System.out.println(
            "Hello from hidden class!"
        );
    }
}
}

```

Important Line

The key operation is:

```

MethodHandles.Lookup hiddenLookup =
    lookup.defineHiddenClass(
        classBytes,
        true
    );

```

This asks the JVM to define the supplied class bytes as a **hidden class**.

Creating and Using a Hidden Class

The returned Lookup object can be used to access the newly defined class:

```
Class<?> hiddenClass =  
    hiddenLookup.lookupClass();  
  
System.out.println(  
    hiddenClass.isHidden()  
);
```

The Class.isHidden() method can be used to determine whether the class is hidden.

A typical result is:

true

Understanding the API

The main API involved is:

java.lang.invoke.MethodHandles.Lookup

The important method is:

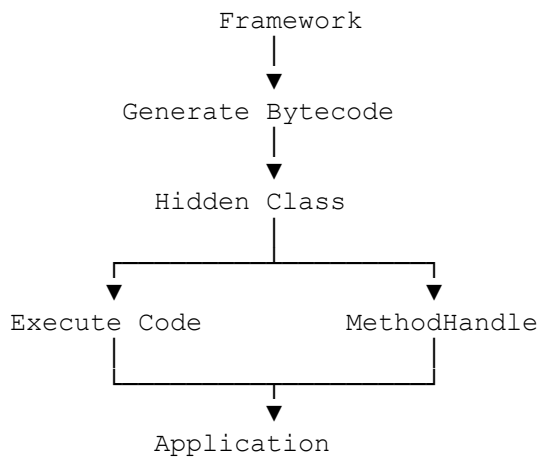
```
defineHiddenClass(  
    byte[] bytes,  
    boolean initialize,  
    ClassOption... options  
)
```

Conceptually:

```
Class File Bytes  
    ↓  
defineHiddenClass()  
    ↓  
Lookup  
    ↓  
Hidden Class  
    ↓  
Class.isHidden() → true
```

Why Frameworks Need Hidden Classes

Consider a framework that generates implementation classes dynamically:



This avoids treating every dynamically generated implementation as a conventional application class.

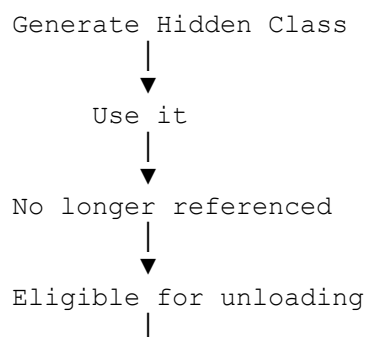
Hidden classes are useful in technologies involving:

- Dynamic proxies
- Lambda implementation machinery
- Bytecode generation
- JVM language runtimes
- Runtime-generated adapters
- Metaprogramming frameworks

Hidden Class and Class Unloading

One important motivation behind hidden classes is their relationship with **class unloading**.

A dynamically generated class may only be needed temporarily:



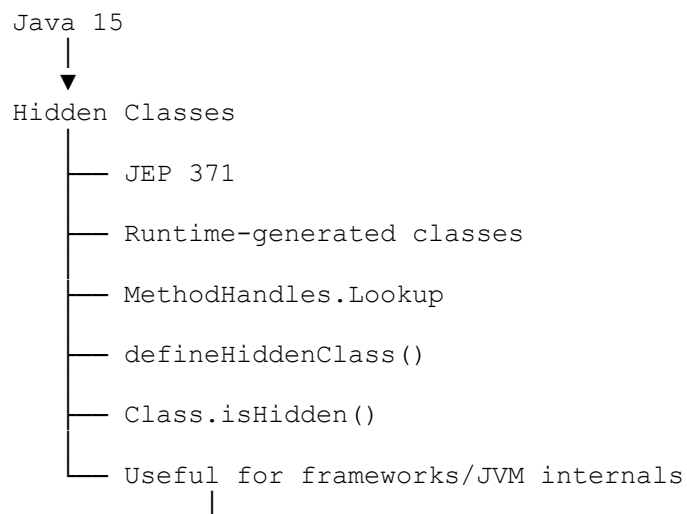
▼
JVM can reclaim
associated metadata

This is particularly useful for systems that generate many short-lived classes dynamically.

Key Characteristics

Characteristic	Hidden Classes
Introduced	Java 15
JEP	JEP 371
Main API	MethodHandles.Lookup
Definition method	defineHiddenClass()
Detect hidden status	Class.isHidden()
Primary users	JVMs, frameworks, runtime code generators
Normal class lookup	Not intended
Runtime generation	Strong use case
Class unloading	Supports efficient unloading of dynamically generated classes

Key Point



In short: Hidden Classes, introduced in Java 15, provide a JVM-supported mechanism for creating classes intended primarily for **runtime-generated code and framework implementation details**, with special lifecycle and discoverability characteristics that make them different from ordinary named classes.

Java 15 — Sealed Classes (Preview)

Java Version: 15

Year: 2020

Feature: Sealed Classes (*Preview*)

UniqueClassName: SealedPreview

Definition

Sealed Classes were introduced as a **preview feature in Java 15** through **JEP 360**.

A sealed class or interface allows a developer to explicitly specify **which classes or interfaces are permitted to extend or implement it**.

The sealed and permits keywords are used to control inheritance.

Basic Syntax

```
public sealed class Shape
    permits Circle, Rectangle {
}
```

This means:

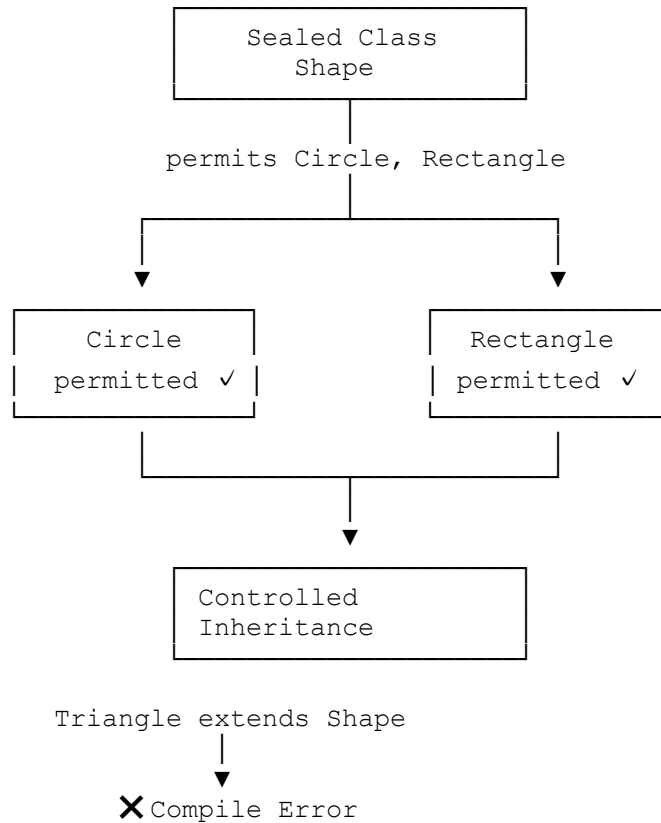
Shape

└─ Circle	✓ permitted
└─ Rectangle	✓ permitted
└─ Triangle	✗ not permitted

The main purpose is to provide **controlled inheritance** and make class hierarchies more predictable.

Important: Sealed classes were a preview feature in Java 15, refined in Java 16, and became a permanent feature in **Java 17**.

Flowchart



Java Code Example

SealedPreview.java

```
public class SealedPreview {  
    public static void main(String[] args) {  
        Shape circle = new Circle(5);  
        Shape rectangle = new Rectangle(10, 20);  
        System.out.println(  
            "Circle area: " + circle.area()  
        );  
        System.out.println(  
            "Rectangle area: " + rectangle.area()  
        );  
    }  
}
```

```

        );
    }
}

// Sealed class
sealed abstract class Shape
    permits Circle, Rectangle {

    abstract double area();
}

// Permitted subclass
final class Circle extends Shape {

    private final double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    @Override
    double area() {
        return Math.PI * radius * radius;
    }
}

// Permitted subclass
final class Rectangle extends Shape {

    private final double width;
    private final double height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    double area() {
        return width * height;
    }
}

```

Output

Circle area: 78.53981633974483

Rectangle area: 200.0

Understanding sealed and permits

The declaration:

```
sealed abstract class Shape
    permits Circle, Rectangle {
```

contains two important parts.

sealed

sealed

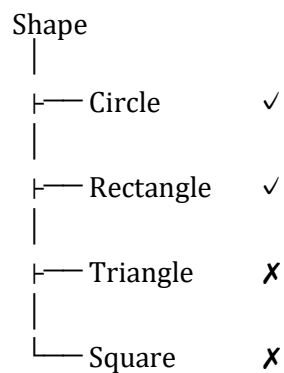
says that the class has a **restricted set of subclasses**.

permits

permits Circle, Rectangle

explicitly lists the classes allowed to extend Shape.

Therefore:



Invalid Subclass Example

Suppose we attempt:

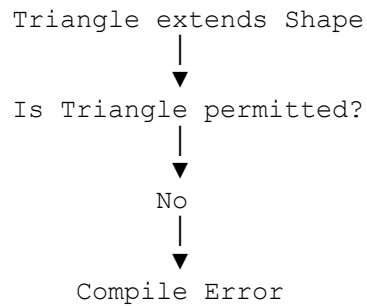
```
class Triangle extends Shape {
    @Override
    double area() {
        return 0;
    }
}
```

Since Triangle is not included in:

permits Circle, Rectangle

the compiler rejects it.

Conceptually:



This provides compile-time enforcement of the intended class hierarchy.

Permitted Classes Must Declare Their Inheritance Policy

A permitted subclass must itself be declared as one of:

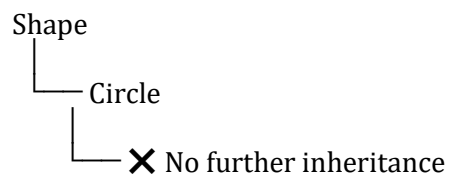
final
sealed
non-sealed

For example:

final

```
final class Circle extends Shape {  
}
```

No class can extend Circle.



sealed

A permitted subclass can continue restricting its own subclasses:

```
sealed class Polygon extends Shape
  permits Triangle, Square {
}
```

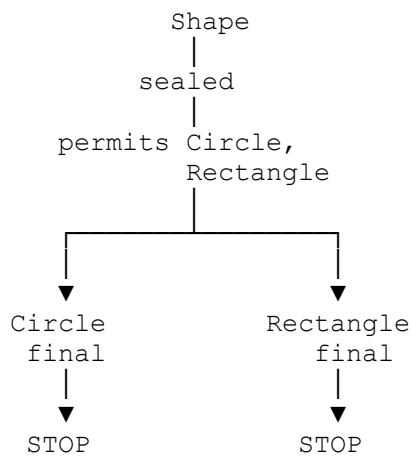
non-sealed

A permitted subclass can remove the inheritance restriction:

```
non-sealed class OpenShape extends Shape {
}
```

Other classes can then extend OpenShape.

Sealed Class Hierarchy



This creates a controlled and explicitly documented inheritance hierarchy.

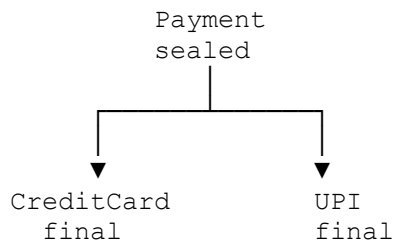
Sealed Interfaces

Sealed types are not limited to classes. Interfaces can also be sealed.

```
sealed interface Payment
  permits CreditCard, UPI {
}
```

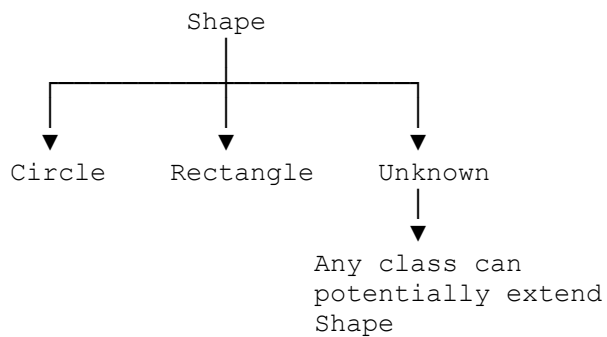
```
final class CreditCard implements Payment {  
}  
  
final class UPI implements Payment {  
}
```

The hierarchy becomes:

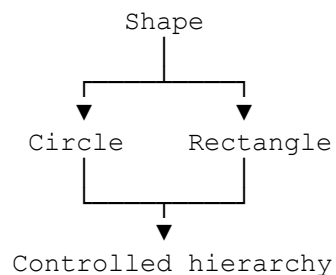


Why Sealed Classes Are Useful

Without sealed classes:



With sealed classes:



This is particularly useful for **domain modeling**, APIs, frameworks, and type hierarchies where the possible implementations should be known and controlled.

Java 15 Compilation

Because sealed classes were a preview feature in Java 15, use:

```
javac --enable-preview --release 15 SealedPreview.java
```

Run with:

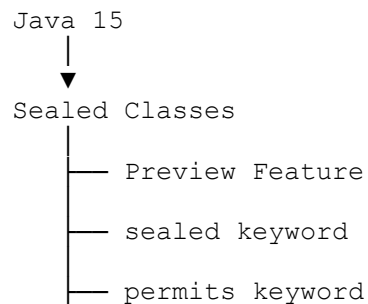
```
java --enable-preview SealedPreview
```

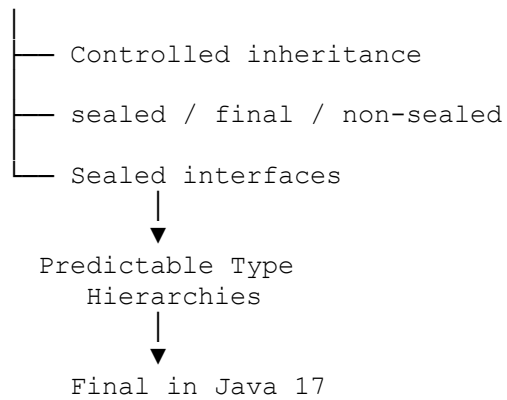
No `--enable-preview` flag is required when using sealed classes in modern Java versions where the feature is permanent.

Sealed Classes vs Normal Classes

Feature	Normal Class	Sealed Class
Inheritance	Generally unrestricted	Explicitly restricted
permits	Not available	Available
sealed	No	Yes
final subclasses	Optional	Required policy for permitted subclasses
Compile-time hierarchy control	Limited	Strong
Java 15	Standard	Preview
Java 17	Standard	Standard

Key Point





In short: Java 15 introduced **Sealed Classes as a preview feature**, allowing developers to explicitly control which classes can extend a class or implement an interface. This makes domain hierarchies more **controlled, predictable, and maintainable**.